Java Exception Handling Interview Questions & Answers

1. What is an Exception in Java?

An exception is an event that disrupts the normal flow of a program. It is an object that is thrown at runtime when an error occurs. Java provides a robust and object-oriented way to handle exceptions using try-catch blocks.

2. What is the difference between Checked and Unchecked Exceptions?

Checked exceptions are checked at compile-time (e.g., IOException, SQLException). Unchecked exceptions are checked at runtime (e.g., NullPointerException, ArithmeticException).

3. What is the difference between throw and throws?

`throw` is used to explicitly throw an exception, while `throws` is used in method signature to declare exceptions that might be thrown.

4. Can we catch multiple exceptions in a single catch block?

Yes, from Java 7 onwards, you can use multi-catch to catch multiple exceptions in a single block using the pipe `|` symbol.

5. What is the purpose of the finally block?

The finally block is used to execute important code such as closing resources. It always executes regardless of whether an exception is thrown or caught.

6. What is the difference between Error and Exception?

`Error` represents serious problems that applications should not try to catch (e.g., OutOfMemoryError). `Exception` represents conditions that applications might want to catch.

7. How does exception propagation work in Java?

When an exception is not caught in the current method, it propagates to the calling method. This continues until the exception is caught or the program terminates.

8. What is exception chaining?

Exception chaining is the practice of wrapping one exception inside another to preserve the original cause. This is done using constructors or `initCause()` method.

9. How do you create a custom exception in Java?

You can create a custom exception by extending the `Exception` class (for checked exceptions) or `RuntimeException` (for unchecked exceptions).

10. What are best practices for exception handling?

Use specific exceptions, avoid empty catch blocks, log exceptions properly, use custom exceptions for domain-specific errors, and avoid using exceptions for control flow.

11. Can we have a try block without a catch block?

Yes, a try block can be followed by a finally block without a catch block. The finally block will execute regardless of exception occurrence.

12. What happens if an exception is thrown in the finally block?

If an exception is thrown in the finally block, it can override any exception thrown in the try or catch block, potentially hiding the original exception.

13. Should custom exceptions be checked or unchecked?

It depends on the use case. Use checked exceptions for recoverable conditions and unchecked exceptions for programming errors.

14. How do you handle exceptions in layered architecture?

Catch exceptions at the appropriate layer, wrap them in custom exceptions if needed, and propagate meaningful messages to the upper layers or user interface.